



Penetration Test Report



Security Assessment & Penetration Test Report

Target System: Apex Accelerator Portal

<https://apex-accelerator-portal.vercel.app> | xfucpvivifshrcagfann.supabase.co

Prepared For: Muhammad Ali Raza Abdi

Prepared By: Syed Shabahat Hussain



Contents

- Executive Summary
- Vulnerability Summary
- Detailed Findings

1 · Executive Summary

A scoped web application penetration test was conducted on the **Apex Accelerator Portal** and its underlying Supabase REST API. The primary objective was to evaluate

the application's security posture and identify vulnerabilities that could lead to unauthorized access, privilege escalation, or data manipulation.

The assessment uncovered **Critical** security flaws stemming from improper authorization enforcement. Notably, the backend lacks adequate **Row Level Security (RLS)** policies. By exploiting Broken Object Level Authorization (BOLA), an unprivileged user can manually patch API endpoints to arbitrarily escalate their privileges to "Admin" and manipulate internal platform credits. Furthermore, high-severity injection flaws (XSS) were identified in the user-facing service request portals.



Immediate remediation is strongly recommended to prevent systemic compromise.

2 · Vulnerability Summary

The following table outlines the vulnerabilities discovered during the assessment, ranked by severity.

Vulnerability Description	Severity	Status
● Broken Object Level Authorization (BOLA) - Privilege Escalation	Critical	Open
● Insecure Direct Object Reference (IDOR) - Financial Manipulation	High	Open
● Stored Cross-Site Scripting (XSS) in Ticket Submission	High	Open
● Information Exposure via Verbose API Errors	Medium	Open
● Improper File Upload Validation / Silent Client Failures	Low	Open

3 · Detailed Findings

● Finding 1: Broken Object Level Authorization (BOLA) — Privilege Escalation



Severity: Critical

Description: The application relies heavily on frontend logic to enforce user roles, failing to properly secure the backend database using Supabase's Row Level Security (RLS). It was discovered that an authenticated, low-privileged user can manually send a **PATCH** request to the **user_roles** table and assign themselves the role of **admin**. This bypasses all intended access controls and grants the attacker full administrative capabilities over the portal.

Steps to Reproduce:

1. Authenticate to the application as a standard user.
2. Intercept web traffic using an interception proxy (e.g., Burp Suite).
3. Send a **PATCH** request to `/rest/v1/user_roles?user_id=eq.[YOUR_USER_ID]` containing the JSON payload `{"role": "admin"}`.
4. Observe the server responds with **HTTP/2 204 No Content**, successfully updating the role.

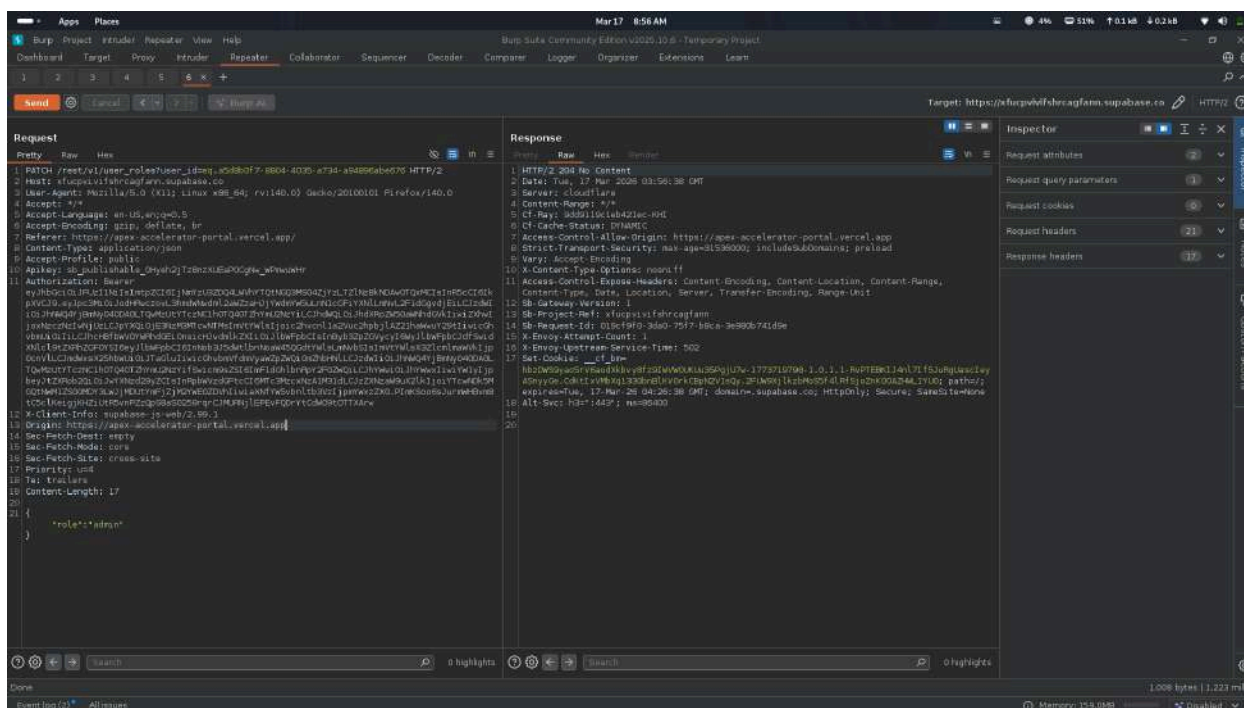


Figure 1: Successful modification of user role to 'admin' via the REST API.



Remediation: Implement strict Row Level Security (RLS) policies on the `user_roles` table. Ensure that policies restrict `UPDATE` operations so that only verified, pre-existing administrators can modify role assignments.

● Finding 2: Insecure Direct Object Reference (IDOR) — Credit Manipulation



Severity: High

Description: Similar to the privilege escalation vulnerability, the `profiles` table lacks sufficient authorization checks for update operations. An attacker can directly manipulate their profile parameters via the API, allowing them to arbitrarily alter platform financial metrics, such as artificially inflating their available credit balance.

Steps to Reproduce:

1. Capture the profile update request using Burp Suite.
2. Modify the request path to target the profiles endpoint: `PATCH /rest/v1/profiles?user_id=eq.[YOUR_USER_ID]`
3. Inject the payload `{"credits": 99999}`.
4. Refresh the dashboard to observe the illicitly obtained credits.

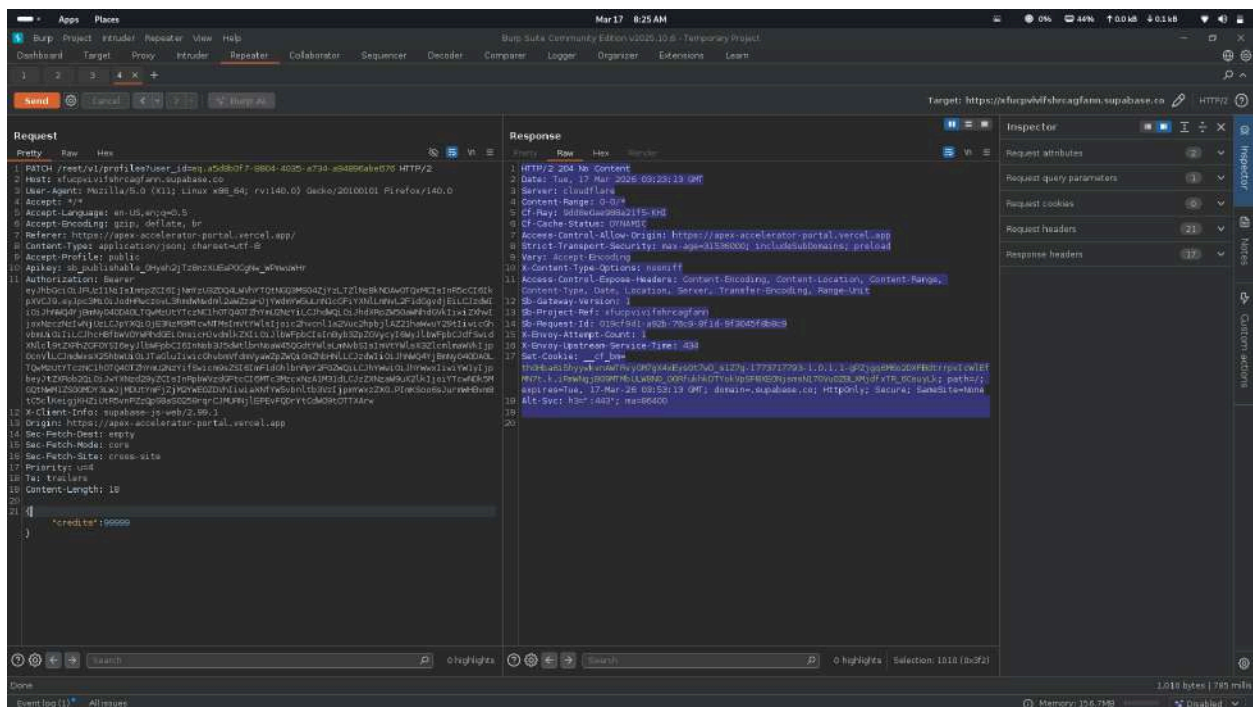


Figure 2: Injecting arbitrary credits directly into the database.

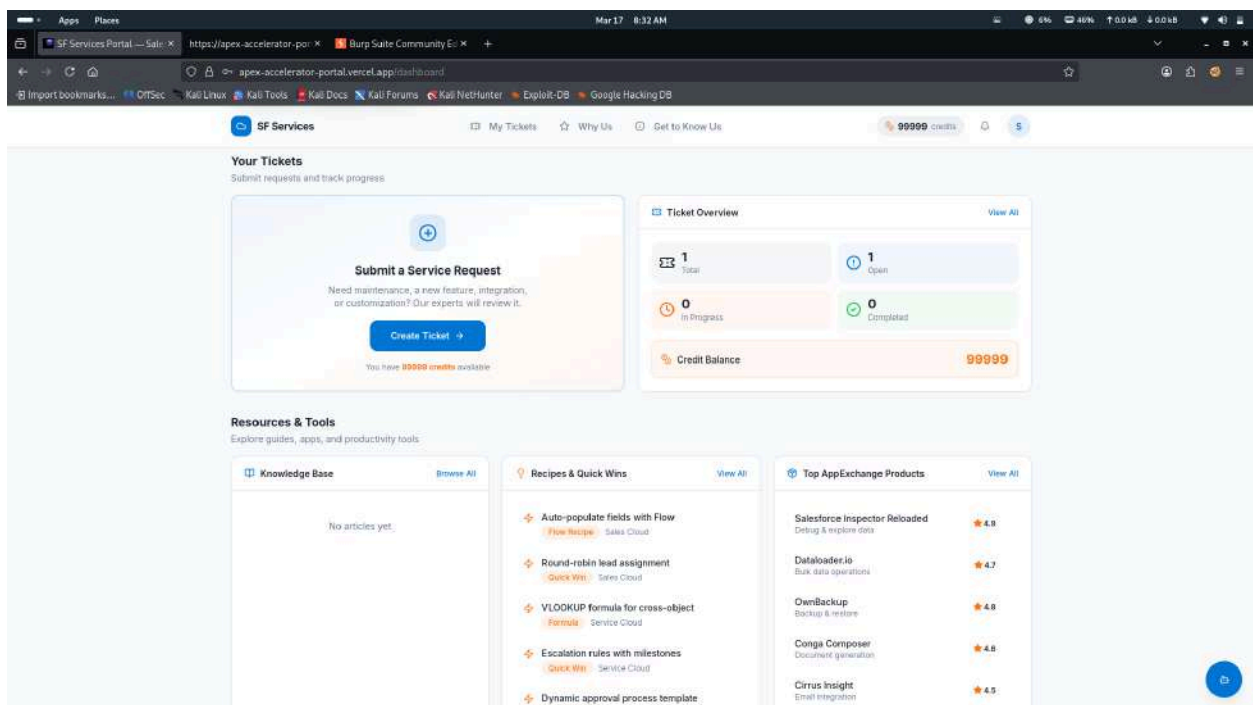


Figure 3: Frontend UI reflecting the manipulated credit balance.



Remediation: Enforce RLS policies on the `profiles` table. Specifically, prevent users from modifying sensitive columns like `credits`. Financial transactions and credit modifications should only be handled via secure, server-side functions that validate payment or administrative authorization.

● Finding 3: Stored Cross-Site Scripting (XSS) in Service Requests



Severity: High

Description: The "Submit a Service Request" module does not adequately sanitize user-supplied input before storing it in the database and reflecting it back to users. If an attacker injects malicious JavaScript into the "Requirements" field, the payload will execute in the browser context of any administrator or support staff member who views the ticket, potentially leading to **session hijacking**.

The screenshot shows Burp Suite's HTTP history and request details. The selected request is a POST to `/api/debug_my_memberships` with a status code of 200. The request body is a JSON object containing a malicious JavaScript payload. The payload is a long string of JavaScript code, including a `<script>` tag and a `</script>` tag, which is designed to hijack a session. The payload is reflected in the response body, indicating that the application does not sanitize user input before reflecting it back to users.

Figure 4: The vulnerable "Requirements" input field during ticket submission.



Remediation: Implement robust context-aware output encoding. Ensure the React frontend utilizes secure DOM-purification libraries (e.g., **DOMPurify**) when rendering user-supplied text from the database to strictly prevent script execution.

● Finding 4: Information Exposure via Verbose API Errors



Severity: Medium

Description: The PostgREST API is configured to return verbose error messages that leak internal database schema structures. Malformed requests yield **PGRST** errors that explicitly suggest valid internal function names and table relationships, aiding an attacker in mapping the backend infrastructure.



Remediation: Suppress verbose database errors in production. Configure the Supabase API gateway settings to return generic, non-descriptive error messages (e.g., standard **400** or **500** status codes) for invalid endpoints.

● Finding 5: Improper File Upload Validation & Silent Failures



Severity: Low

Description: The ticket attachment functionality fails silently on the client side when attempting to upload unexpected file types. While the frontend interface silently drops the upload, the backend API lacks strict verification of MIME types and file extensions, relying primarily on weak frontend restrictions.



Remediation: Implement strict server-side validation to enforce allowed MIME types (e.g., PDF, PNG, JPG) and enforce file size limits. Additionally, improve frontend error handling to provide clear feedback when an upload is rejected to improve user experience.